

Documentación Técnica Completa de Arquitectura y Despliegue - CIIN Diagnósticos

Este documento constituye el manual técnico definitivo y milimétrico del sistema digital integrado para **CIIN Diagnósticos**. Detalla la arquitectura de software desacoplada, las configuraciones del sistema, el catálogo completo de componentes de la interfaz de usuario, los flujos de red en el servidor Apache de Telmex, las estrategias de resiliencia implementadas en el Front-End y Back-End, y la arquitectura interna de Laravel.

1. Fundamentos de la Arquitectura y Justificación Tecnológica

El sistema adopta un modelo de desarrollo desacoplado compuesto por un Front-End estructurado como una Single Page Application (SPA) y un Back-End robusto que opera como una API RESTful. La elección y convivencia de estas tecnologías responde a restricciones físicas y operativas específicas del entorno de producción:

- **Restricción del Host (Telmex Nomad / Carrierzone):** El entorno de servidor compartido provisto está optimizado nativamente para ejecutar entornos basados en PHP (actualmente configurado para interpretar PHP 8.4-CGI). Debido a esto, se determinó utilizar el framework **Laravel** para gobernar el núcleo del Back-End, garantizando un manejo profesional de rutas, ORM, colas de procesamiento y conexiones seguras a bases de datos relacionales sin requerir un entorno VPS dedicado.
- **Experiencia de Usuario Fluida (React SPA):** Para la capa de presentación se seleccionó **React**. Compilar la interfaz de usuario en archivos estáticos HTML, CSS y JavaScript permite que el navegador del cliente cargue la aplicación una sola vez, interactuando de forma asíncrona mediante peticiones HTTP con el Back-End. Esto elimina los parpadeos de recarga de página tradicionales y reduce el consumo de transferencia de datos en el hosting.

2. Configuración Avanzada del Servidor Apache (.htaccess)

Para posibilitar la coexistencia de la SPA de React (en la raíz pública del servidor) y el motor de

Laravel (alojado de forma segura en el directorio secundario

core/

), el archivo de configuración perimetral de Apache

.htaccess

ha sido programado de forma milimétrica. Este archivo intercepta cada petición entrante y decide qué componente debe procesarla.

RewriteEngine On

RewriteBase /

1. POLÍTICA DE SEGURIDAD: FORZAR HTTPS

RewriteCond %{HTTPS} !=on

RewriteRule ^(.*)\$ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301,NE]

2. MECANISMO ROMPE-BUCLES INTERNOS

RewriteRule ^core/public/index\.php\$ - [L]

3. ENRUTAMIENTO EXCLUSIVO DE LA API HACIA LARAVEL

RewriteCond %{REQUEST_URI} ^/api [NC]

RewriteRule ^api/(.*)\$ core/public/index.php [L,QSA]

4. PROTECCIÓN DEL ENTORNO CORE

RewriteCond %{REQUEST_URI} ^/core [NC]

RewriteRule ^core/(.*)\$ core/public/index.php [L,QSA]

5. ENRUTAMIENTO VIRTUAL PARA REACT SPA

RewriteCond %{REQUEST_FILENAME} !-f

RewriteCond %{REQUEST_FILENAME} !-d

RewriteCond %{REQUEST_URI} !^/api [NC]

RewriteCond %{REQUEST_URI} !^/core [NC]

RewriteRule ^ index.html [L]

6. CONFIGURACIÓN DEL HANDLER CGI DE TELMEX

AddHandler php-cgi .php

Action php-cgi /cgi-bin/php-cgi

3. Front-End: Stack Tecnológico, Arquitectura y UI

(React)

La interfaz gráfica implementa un sistema de diseño consistente enfocado en el sector salud e innovación digital. Se construyó utilizando la biblioteca **React.js** junto con un ecosistema robusto de dependencias para garantizar interactividad, seguridad y diseño adaptativo.

3.1 Librerías y Dependencias Principales

- **React-Router-Dom:** Encargado del enrutamiento virtual de la SPA. Define las rutas (`/contacto`, `/blog`, `/diagnostico`) e implementa componentes anidados usando `Outlet` en el layout principal.
- **React-Bootstrap:** Framework de diseño principal. Utilizado para estructurar los layouts (`Container`, `Row`, `Col`), componentes interactivos (`Carousel`, `Navbar`) y formularios controlados.
- **React-Icons:** Proveedor de iconografía vectorial ligera (ej. `FaFacebook`, `FaCheck`, `FaTimesCircle`).
- **@marsidev/react-turnstile:** Componente oficial para la integración del widget de Cloudflare Turnstile, esencial para la protección contra bots en el formulario de diagnóstico.

3.2 Estructura de Ruteo y Layout HOC (ExportPrin.js)

El sistema de navegación está centralizado en

`App.js`

, que delega el renderizado al componente contenedor

`ExportPrin`

. Este último actúa como un *Higher-Order Component (Layout)* que engloba a todas las vistas.

Incluye el

`NavBar`

superior, el

`Footer`

inferior y gestiona una animación global de transición mediante un estado

`animar`

y el hook

`useLocation`

. Cada vez que la ruta cambia, se dispara un círculo expansivo (

`.circle.animate`

) para suavizar el cambio de vista.

3.3 Desglose Pantalla por Pantalla

1. Inicio (Landing Page):

- Incorpora un efecto **Parallax** personalizado calculando la posición del scroll (`window.pageYOffset`) para desvanecer y trasladar (`translateY`) el carrusel principal.
- Muestra tarjetas de servicios con iconos vectoriales y un iframe incrustado de Google Maps.

2. Diagnóstico (/diagnostico):

- Implementa un formulario complejo con diseño tipo **Glassmorphism** (`backdrop-filter: blur(20px)`).
- El formulario es controlado mediante un único estado en forma de objeto (`datosForm`).
- Ejecuta validaciones estrictas en el lado del cliente (ej. confirmación de correo) antes de emitir la petición `fetch` con el token de seguridad de Turnstile.

3. Contacto (/contacto):

- Formulario asíncrono para atención directa, estilizado con campos limpios que brillan al enfoque (`.input-verde:focus`). Integra el componente dinámico `ModalCarga`.

4. Módulo de Blog (/blog y /blog-detalle):

- **Animaciones en Scroll:** Utiliza el API nativa `IntersectionObserver` combinada con `useRef` para añadir la clase `show` a los artículos y videos solo cuando entran en la ventana de visión del usuario, mejorando el rendimiento.
- **Manejo de Estados Vacíos:** Renderiza un marcador visual (placeholder) si el Backend responde con un arreglo vacío.
- **Navegación con Estado:** Utiliza el hook `useLocation` de React Router para pasar el objeto del artículo completo desde el índice del blog hacia la vista de detalle (`blogDetalle.js`), evitando hacer otra petición extra a la base de datos.

3.4 Componentes Críticos

- **NavBar Interactivo:** Posee un efecto de "revelado" en hover con retardo (``setTimeout`` en ``onMouseEnter``), mostrando texto animado a la derecha del logo en resoluciones de escritorio, y colapsando a un menú inferior en dispositivos móviles con anulación de fondos SVG.
- **ModalCarga:** Componente altamente dinámico renderizado condicionalmente. Para la vista de Diagnóstico, ejecuta dos temporizadores (``setInterval``): uno para añadir puntos suspensivos ("...") y otro para rotar iterativamente mensajes de estado ("Analizando solicitud", "Generando respuesta"), simulando actividad de inteligencia artificial.

3.5 Sistema Visual y Animaciones Custom

Clase CSS / Elemento	Efecto o Propiedad Técnica
.animacionFondo	Sistema de partículas flotantes (burbujas). Utiliza el hook <pre>useMemo</pre> para inyectar variables CSS nativas (<pre>--dim</pre> , <pre>--posX</pre>) creando un efecto visual aleatorio y no bloqueante.
.circle.animate	Transición de páginas mediante un círculo <pre>#41d2b8</pre> que se expande a <pre>250vmax</pre> y luego colapsa fuera de la pantalla.

Clase CSS / Elemento	Efecto o Propiedad Técnica
.navText (Degradados)	Texto transparente con un fondo de gradiente animado mediante <pre>background-clip: text</pre> y <pre>background-position</pre> .

4. Back-End: Arquitectura API RESTful en Laravel

El sistema de Back-End está diseñado en base al patrón MVC (Modelo-Vista-Controlador), exponiendo servicios a través de rutas API puras. Todas las peticiones están mediadas y validadas.

4.1 Seguridad de Acceso: El Bearer Token (Sanctum)

La API está completamente cerrada y blindada al público general mediante el paquete **Laravel Sanctum**. El flujo de validación opera de la siguiente manera:

- **Rutas Protegidas:** En el archivo `routes/api.php`, todos los endpoints operativos están envueltos en el grupo `middleware` `['auth:sanctum']`.
- **Peticiones Front-End:** React debe inyectar el Header HTTP: `Authorization: Bearer {REACT_APP_TOKEN_API}` en cada solicitud `fetch`.
- **Validación Base de Datos:** Cuando la petición llega a Laravel, Sanctum encripta el token recibido mediante algoritmo SHA-256 y lo compara contra la tabla `personal_access_tokens`. Si la huella no coincide o el token expiró, la petición es rechazada inmediatamente con un error `401 Unauthorized`, evitando el procesamiento del controlador y protegiendo la base de datos de accesos no autorizados.

4.2 Modelos y Migraciones (ORM Eloquent)

La arquitectura de datos se define a través de migraciones estrictas y modelos de Eloquent configurados con

```
$fillable
```

para prevenir asignaciones masivas de datos vulnerables:

- **Modelo**

Cliente: Almacena los perfiles empresariales procesados, definiendo atributos rigurosos de negocio (

```
nom_empresa,
```

```
rubro_empresa,
```

```
problematica). Se implementó
```

```
firstOrCreate
```

 en el controlador para mantener la integridad e impedir registros empresariales duplicados basados en el correo y nombre corporativo.

- **Modelo**

Blog: Administra la bitácora de artículos, soportando campos extensos para HTML enriquecido (

```
cont_blog
```

 max 10000 chars) y un flag booleano (

```
esDestacado
```

) para la priorización en el UI.

- **Modelo**

Contacto: Actúa como buzón relacional ligero para la recepción de sugerencias generales y soporte técnico.

4.3 Controladores y Catálogo de Endpoints

La lógica de negocio está segregada en controladores específicos que aplican validaciones de Request previas a la ejecución lógica:

Endpoint API	Controlador	Lógica Principal
/api/blog (GET/POST/PUT)	BlogsController	Administra artículos. Si la colección de la BD está vacía, devuelve <pre>404 Not Found</pre> con un mensaje explícito capturado por el catch de React. Gestiona logs de error en operaciones de

Endpoint API	Controlador	Lógica Principal
		escritura.
/api/contactos (POST)	ContactoController	Crea el registro de sugerencia, invoca a la clase <code>MensajeSugerencia</code> y encola el envío de correo hacia la administración corporativa de forma síncrona.
/api/clientes (POST)	ClienteController	El orquestador principal. Valida Token Turnstile, guarda el prospecto, invoca al <code>GeminiService</code> bajo ciclo while de 5 reintentos y despacha dos Mailable views independientes (<code>CorreoDiagnostico</code> para el cliente, <code>CorreoJefe</code> para ventas).

5. Arquitectura del Servicio de Inteligencia Artificial (Gemini)

El sistema de IA no reside en un controlador genérico, sino que está abstraído dentro de

`App\Services\GeminiService`

, aplicando el principio de responsabilidad única. La comunicación se realiza mediante el Facade

`Http`

contra la API

`gemini-2.5-flash`

.

5.1 Estructura del Prompt Engineering

El

`prompt`

enviado al LLM está diseñado algorítmicamente para reducir la "alucinación" de la IA y asegurar respuestas concisas y directas:

- **RoI Definido:** "Consultor Senior de Negocios con 30 años de experiencia".
- **Inyección Dinámica de Contexto:** Interpola las variables purificadas de `$cliente` en los campos de "Tamaño de equipo", "Rubro", y "Problema central".
- **Restricciones de Salida Estrictas:** Se le exige al modelo que omita el formato Markdown (
*,
**), usando saltos de línea crudos, con un límite riguroso de cuatro párrafos y tono ejecutivo corporativo.

6. Arquitectura de Plantillas de Correo (Mailable Views)

Las vistas HTML enviadas por el servicio SMTP/Sendmail están codificadas en la carpeta

`resources/views/emails`

. Para garantizar la compatibilidad con clientes de correo estandarizados e históricos (como Outlook de escritorio), las plantillas están arquitectadas utilizando tablas HTML anidadas (

`<table><tr><td>`

) en lugar de CSS Flexbox o Grid.

- **`emails.diagnostico`:** Dirigido al usuario final. Incorpora el logotipo institucional (usando `$message->embed()`), presenta el texto derivado de la IA en una caja estilizada y formatea los saltos de línea de Gemini usando `{!! nl2br(e($diagnostico)) !!}` previniendo inyección XSS.
- **`emails.jefe`:** Dirigido a la fuerza de ventas interna. Expone una tarjeta resumen del

Lead ("Radiografía empresarial") con todos los datos clave de perfilamiento, marcando el correo como un enlace interactivo y exponiendo los "dolores" del prospecto de forma clara para cerrar la venta.

- `emails.sugerencia`: Formato ejecutivo minimalista que captura e informa sobre tickets de soporte originados en el apartado `/contacto`.

7. Seguridad Perimetral y Protección Financiera de la API (Gemini)

Dado que el endpoint de procesamiento de IA (

`/api/clientes`

) realiza conexiones salientes a la infraestructura comercial de Google Gemini, se desplegó un triple blindaje:

- **Validación Turnstile Server-Side:** El Back-End verifica el `turnstile_token` enviado por React cruzándolo con la API oculta de Cloudflare (`/siteverify`). Si falla, bloquea con código 403.
- **Rate Limiting (Throttle):** El middleware `throttle:limitar-gemini` en el grupo de rutas protege la facturación restringiendo el número de diagnósticos permitidos por IP y devolviendo el código HTTP **429 Too Many Requests** ante ráfagas DDoS.
- **Reintentos y Cierres (While-Loop Resiliencia):** Si los servidores de Gemini se caen o están saturados (503 Service Unavailable), el Back-End posee un bucle `while` que fuerza un `sleep(2)` y reintentla la conexión hasta 5 veces antes de interrumpir la orden.

8. Capa de Mensajería Transaccional e Ingeniería de Redes (Bypass de Firewall)

Para evadir el bloqueo de hardware sobre puertos salientes SMTP (25, 465 y 587) aplicado por Telmex, se implementó el protocolo

`MAIL_MAILER=sendmail`

. Laravel enruta los correos a la cola interna de Linux, de donde son despachados hacia la red mundial por los Agentes de Transferencia (MTA) del servidor, previniendo caídas de comunicación y bloqueos Firewall de nivel externo.

9. Protocolos Seguros de Despliegue Operativo

El sistema estandariza el uso del cliente FTP **FileZilla** para mitigar limitaciones del panel Web de Carrierzone/Nomad. Ante caídas sistémicas en herramientas MySQL visuales nativas (

```
mysqli::real_connect() : (HY000/2002)
```

), se instrumenta el despliegue transitorio de Adminer, el cual debe ser destruido obligatoriamente en disco para garantizar la higiene del servidor.